

CURSO DE PROGRAMACIÓN

NIVEL 1 · TEORÍA PARA PRINCIPIANTES

Una guía para comprender los conceptos básicos de la programación y aprender Python paso a paso, sin ejercicios ni examen.

Requisitos: no necesitas experiencia previa. Los ejemplos de código sirven para ilustrar las explicaciones; no hay tareas que tengas que completar.

Material introductorio · Edición 2026

Cómo leer este curso

Este documento está pensado para aprender leyendo y entendiendo los conceptos. Cada módulo explica una idea y muestra pequeños fragmentos de Python para que puedas ver cómo se escribe el código. No contiene ejercicios, actividades ni examen.

Lo que aprenderás

• Qué es programar y cómo se estructura un programa.

• Cómo escribir instrucciones básicas en Python.

• Qué son las variables y los tipos de datos.

• Cómo recibir información y realizar cálculos.

• Cómo funcionan las condiciones y los bucles.

• Cómo organizar código con listas y funciones.

• Cómo interpretar errores y seguir buenas prácticas.

Índice

1. Introducción a la programación

2. Python y las primeras instrucciones

3. Variables y tipos de datos

4. Entrada de datos y operadores

5. Condiciones

6. Bucles

7. Listas y funciones

8. Errores y buenas prácticas

9. Glosario y próximos pasos

Idea importante: programar no consiste en memorizar todo el código, sino en comprender cómo se divide un problema en instrucciones claras.

1. Introducción a la programación

Programar significa escribir instrucciones para que un ordenador, teléfono u otro dispositivo realice una tarea. Esas instrucciones pueden servir para mostrar una página web, ejecutar un videojuego, calcular resultados o gestionar información.

Conceptos fundamentales

• **Algoritmo:** conjunto ordenado de pasos para resolver un problema.

• **Lenguaje de programación:** sistema de reglas para escribir instrucciones que un ordenador pueda ejecutar, como Python, JavaScript o Java.

• **Código fuente:** texto que escribe una persona para indicar al programa qué debe hacer.

• **Programa:** conjunto de instrucciones que se ejecutan para realizar una tarea.

• **Depuración:** proceso de localizar y corregir errores en el código.

Cómo se construye un programa

Normalmente se empieza identificando qué se quiere conseguir. Después se divide el objetivo en pasos pequeños, se escribe el código, se ejecuta y se revisa el resultado. Si algo no funciona como se esperaba, se busca el error y se corrige.

Lenguajes de programación conocidos

Python es popular para aprender, automatizar tareas y trabajar con datos. **JavaScript** se utiliza mucho para añadir interactividad a páginas web. **Java** se usa en distintos tipos de aplicaciones y sistemas. Cada lenguaje tiene sus usos y reglas.

> En este curso se utiliza Python porque su sintaxis es relativamente sencilla para empezar.

2. Python y las primeras instrucciones

Python es un lenguaje de programación de propósito general. El código se escribe en un editor y luego se ejecuta con Python. Si no quieres instalar nada, puedes utilizar un entorno de aprendizaje en línea de confianza.

La instrucción print()

print() muestra información en pantalla. El texto que quieres mostrar se escribe entre comillas.

```
print("Hola, mundo")
```

Al ejecutar esa línea, aparece el mensaje Hola, mundo. Los paréntesis indican qué información se pasa a la función print().

Mostrar varias cosas

```
print("Estoy aprendiendo Python")
print(2 + 3)
```

La primera línea muestra texto y la segunda calcula una suma y muestra el resultado. Python puede trabajar tanto con texto como con valores numéricos.

Mayúsculas, minúsculas y símbolos

Python distingue las mayúsculas de las minúsculas: print y Print no son lo mismo. Las comillas, los paréntesis, los dos puntos y la indentación tienen funciones concretas. Un símbolo mal escrito puede impedir que el programa se ejecute.

Un programa se ejecuta siguiendo sus instrucciones en orden, salvo que una condición, un bucle o una llamada a una función cambie ese flujo.

3. Variables y tipos de datos

Una variable es un nombre asociado a un valor. Sirve para conservar información y utilizarla más adelante. El valor puede cambiar durante la ejecución del programa.

```
nombre = "Alex"
edad = 14
altura = 1.60
activo = True
```

En este ejemplo, nombre guarda texto, edad un entero, altura un decimal y activo un valor lógico. Los datos son ficticios.

Tipos de datos principales

Tipo	Qué representa	Ejemplo
str	Texto	"Hola"
int	Número entero	14
float	Número decimal	3.5
bool	Verdadero o falso	True / False

Asignar y cambiar valores

```
puntos = 10
puntos = puntos + 5
print(puntos)
```

El signo = asigna un valor a una variable. En la segunda línea se toma el valor actual de puntos, se le suma cinco y se guarda el nuevo resultado. El programa muestra 15.

Los nombres de variables deberían ser claros, por ejemplo puntuacion_total. No se recomienda usar nombres confusos ni palabras reservadas del lenguaje.

4. Entrada de datos y operadores

La función input() permite recibir texto escrito por una persona mientras se ejecuta el programa.

```
nombre = input("¿Cómo te llamas? ")
print("Hola,", nombre)
```

El programa muestra una pregunta, espera la respuesta y la guarda en la variable nombre. Después presenta un saludo.

Convertir texto en números

Aunque una persona escriba dígitos, input() devuelve texto. Para realizar cálculos hay que convertirlo a un tipo numérico, por ejemplo con int() para enteros o float() para decimales.

```
edad = int(input("¿Cuántos años tienes? "))
print(edad + 1)
```

Este programa convierte la respuesta a un entero y muestra el resultado de sumarle uno. Si se introduce un valor que no se puede convertir, puede aparecer un error.

Operadores aritméticos

Símbolo	Operación	Ejemplo
+	Suma	8 + 2 = 10
-	Resta	8 - 2 = 6
*	Multiplicación	8 * 2 = 16
/	División	8 / 2 = 4.0
**	Potencia	2 ** 3 = 8

5. Condiciones

Las condiciones permiten que un programa tome decisiones. El código comprueba si una expresión es verdadera o falsa y, según el resultado, ejecuta unas instrucciones u otras.

```
puntos = 75

if puntos >= 50:
    print("Reto superado")
else:
    print("Sigue intentándolo")
```

if introduce la condición. Si se cumple, se ejecuta el bloque indentado que aparece debajo. else define qué hacer cuando la condición no se cumple. elif permite comprobar una condición adicional.

Operadores de comparación

• **==** comprueba si dos valores son iguales.

• **!=** comprueba si son diferentes.

• **>** significa mayor que y **<** significa menor que.

• **>=** significa mayor o igual que y **<=** menor o igual que.

Combinar condiciones

and exige que ambas condiciones sean verdaderas. **or** permite que una de las condiciones sea verdadera. **not** invierte el valor lógico.

Es importante distinguir = de ==. El primero asigna un valor; el segundo compara dos valores. Confundirlos es un error común al empezar.

6. Bucles

Un bucle repite un bloque de código. Resulta útil cuando se quiere realizar la misma operación varias veces sin escribir la misma instrucción una y otra vez.

El bucle for

```
for numero in range(1, 6):
    print(numero)
```

Este ejemplo muestra los números del 1 al 5. range(1, 6) genera una secuencia que empieza en 1 y se detiene antes de llegar a 6.

El bucle while

```
contador = 1
while contador <= 3:
    print(contador)
    contador = contador + 1
```

while repite las instrucciones mientras su condición sea verdadera. En este ejemplo, el contador aumenta en cada vuelta hasta que la condición deja de cumplirse.

Cuidado con los bucles infinitos

Si la condición de un while nunca se vuelve falsa, el programa puede repetir las instrucciones sin detenerse. Para evitarlo, hay que revisar cómo cambia la condición en cada vuelta.

Diferencia principal: for se usa con frecuencia para recorrer una secuencia o repetir un número conocido de veces; while se usa cuando la repetición depende de una condición.

7. Listas y funciones

Las listas permiten almacenar varios valores en una sola estructura. Los elementos tienen posiciones, llamadas índices, que en Python empiezan por cero.

```
juegos = ["Aventura", "Carreras", "Puzles"]
print(juegos[0])
juegos.append("Estrategia")
print(len(juegos))
```

juegos[0] obtiene el primer elemento. append() añade un nuevo elemento al final de la lista y len() devuelve cuántos elementos contiene.

Qué es una función

Una función agrupa instrucciones para poder utilizarlas varias veces. Se define con def, puede recibir datos llamados parámetros y puede devolver un resultado.

```
def saludar(nombre):
    print("Hola,", nombre)

saludar("Alex")
```

En este ejemplo, nombre es un parámetro. Cuando se llama a saludar("Alex"), la función utiliza Alex como valor y muestra un saludo.

La instrucción return

```
def sumar(a, b):
    return a + b

resultado = sumar(4, 6)
print(resultado)
```

return devuelve un resultado al lugar donde se llamó la función. En este caso, la función sumar recibe dos valores, los suma y devuelve 10.

Las funciones ayudan a mantener el código ordenado, evitar repeticiones y dividir programas grandes en partes fáciles de entender.

8. Errores y buenas prácticas

Los errores son parte normal de programar. Cuando un programa no funciona, el mensaje de error suele indicar el tipo de problema y, a menudo, la línea donde se detectó.

Errores frecuentes

• **SyntaxError:** el código no sigue las reglas de escritura del lenguaje.

• **NameError:** se utiliza un nombre que Python no reconoce, por ejemplo una variable que no se ha definido.

• **TypeError:** se intenta realizar una operación con tipos de datos incompatibles.

• **ValueError:** un valor tiene un formato que no se puede convertir como se esperaba.

Cómo leer un error

Lee el mensaje con calma, localiza la línea mencionada y comprueba las comillas, paréntesis, nombres de variables, tipos de datos e indentación. A veces el error se encuentra justo antes de la línea que el mensaje señala.

Buenas prácticas

• Usa nombres descriptivos para variables y funciones.

• Mantén la indentación consistente.

• Divide el programa en partes pequeñas y comprensibles.

• Añade comentarios cuando ayuden a explicar una decisión.

• Revisa el código y comprueba cómo se comporta con distintos datos.

• No incluyas contraseñas, claves privadas ni información personal sensible dentro del código.

La programación mejora con la práctica, la lectura de código y la capacidad de entender los errores. No hace falta memorizar cada función: es más importante saber qué hace y cuándo utilizarla.

9. Glosario y próximos pasos

Algoritmo: Pasos ordenados para resolver un problema.

Bug: Error que hace que un programa falle o se comporte de forma inesperada.

Bucle: Estructura que repite instrucciones.

Código fuente: Instrucciones escritas por el programador.

Condición: Expresión que se evalúa como verdadera o falsa.

Función: Bloque de código reutilizable que realiza una tarea.

Índice: Posición de un elemento dentro de una secuencia; en las listas de Python comienza en 0.

Variable: Nombre asociado a un valor.

Python: Lenguaje de programación de propósito general.

Depuración: Proceso de encontrar y corregir errores.

Qué aprender después

Cuando comprendas estos conceptos, puedes continuar con archivos, gestión de errores mediante try/except, módulos y programación orientada a objetos. También puedes aprender HTML y CSS para estructurar y diseñar páginas web, y JavaScript para añadirles interactividad.

Más adelante, el conocimiento de Python puede servir como base para automatización, análisis de datos y desarrollo de aplicaciones. Para videojuegos, puedes estudiar motores como Godot o Unity y aprender el lenguaje que corresponda.

Resumen final

Un programa está formado por instrucciones. Las variables guardan datos; las condiciones permiten tomar decisiones; los bucles repiten tareas; las listas agrupan valores; y las funciones organizan el código. Estos fundamentos aparecen en muchos lenguajes y son una buena base para seguir aprendiendo.

Fin del curso de teoría · Nivel 1